

SolarSight — Constrained Object Detection & Reasoning API

Pre-Hackathon Screening — RAP AI/ML

1. Domain & Dataset

SolarSight performs solar panel fault detection using RT-DETR-L. It detects six domain-specific classes: *Bird Drop, Defective, Dusty, Non Defective, Physical Damage, Snow*.

Dataset Source: [Solar Panel Fault Dataset New (v2)](<https://universe.roboflow.com/6rianstorm/solar-panel-fault-dataset-new>).

Preprocessing: Source annotations contained a mixture of detection and polygon annotations. Polygon annotations were converted to axis-aligned bounding boxes for the detection pipeline. We did not originally create the dataset.

Final Image Split (8,730 total): Train: 7,669 | Validation: 611 | Test: 450. The dataset retains all six classes and includes domain-specific solar fault categories beyond standard COCO object categories.

2. Model Training

- **Model:** RT-DETR-L (Starting point: pretrained `rtdetr-l.pt`)
- **Epochs:** 20 | **Batch Size:** 8 | **Image Size:** 640 | **Seed:** 42
- **Optimizer:** AdamW (Original run used `optimizer=auto`, which resolved to AdamW)
- **Hardware:** Tesla T4 GPU | **Training Time:** ~3.066 hours
- **Environment:** Ultralytics 8.4.150
- **Final Checkpoint:** `model/best.pt`

3. Evaluation

Test performance is lower than validation performance. This indicates a real generalization limitation on the held-out set, which is presented honestly here.

Split	Precision	Recall	mAP50	mAP50-95
Validation	0.529	0.560	0.511	0.348
Held-Out Test	0.442	0.514	0.424	0.271

4. Failure Analysis

1. **Bird Drop:** Test recall is 24.7%. Small-object/background confusion is a likely contributing factor. *Future mitigation:* Higher-resolution inference/training or tiled detection (SAHI).
2. **Defective vs Physical Damage:** Real-world qualitative testing produced a strong Physical Damage detection together with overlapping Defective detections. This demonstrates semantic ambiguity between visually related fault categories. *Future mitigation:* Better class-specific examples/hard negatives and taxonomy-aware post-processing.
3. **Dusty:** Test precision is 34.2%. Visually ambiguous regions can produce lower-confidence Dusty predictions. *Future mitigation:* More lighting/glare variation and hard-negative examples.
4. **Snow:** Test mAP50 is 34.2%; mAP50-95 is 19.7%. Axis-aligned boxes derived from polygon annotations can limit localization quality. *Future mitigation:* Improved box annotations or segmentation-aware labeling.
5. **Non Defective:** Test precision is 37.4%. Healthy-panel regions are semantically difficult to distinguish from visually similar fault/background regions. *Future mitigation:* Stronger hard-negative sampling and clearer class definition.

5. Constrained Reasoning API

The reasoning layer is implemented with ordinary Python logic and explicitly does not use LangChain, LangGraph, CrewAI, AutoGen, or similar agent orchestration frameworks. It operates on structured detector outputs rather than inventing visual facts.

Architecture:

Natural-language question → Intent routing → Detector only when visual evidence is required → Structured detections → Deterministic constrained reasoning → Confidence guardrail

Guardrail Behavior: If detector evidence is insufficient, the system explicitly returns:

"Insufficient information from the detector to answer reliably."

6. API / Engineering Reproducibility

The API was verified through a real running Uvicorn server using raw HTTP requests, not merely mocked unit tests.

- GET /health: HTTP 200
- POST /detect: HTTP 200 (real RT-DETR inference → structured detections returned)
- POST /reason (**visual question**): HTTP 200 (detector invoked → reasoning generated from detections)
- POST /reason (**general question**): HTTP 200 (detector bypassed)
- POST /reason (**insufficient evidence**): HTTP 200 (guardrail triggered → system explicitly refused to guess)
- **Invalid requests**: HTTP 400 / HTTP 422 as appropriate

Test Suite: Unit tests: 14 passed, 1 skipped. Real-model integration test: 1 passed (4.92 seconds using CPU fallback on the latest execution).

7. Reproducibility & Deployment

The repository includes `training/train.py`, `evaluation/evaluate.py`, `requirements.txt`, `README.md`, automated tests, and the `model/best.pt` checkpoint. The repository includes a `Dockerfile` and was statically reviewed for deployment readiness; local Docker runtime execution was not performed because Docker was unavailable on the development host.

8. Limitations & Conclusion

SolarSight demonstrates a reproducible ML pipeline with measurable held-out evaluation, explicit failure analysis, and constrained reasoning secured by a confidence guardrail. It is structured as a production-oriented API and avoids inventing visual hallucination by strictly reasoning from verified bounding boxes.